

Newtonsoft.Json JSONPath Tutorial

Overview

JSONPath is a query language for JSON, similar to XPath for XML. Newtonsoft.Json (Json.NET) provides robust support for JSONPath expressions through the `SelectToken()` and `SelectTokens()` methods.

Installation

```
bash
```

```
Install-Package Newtonsoft.Json
```

Basic Setup

```
csharp
```

```
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;

// Sample JSON data
string jsonString = @"{
  'store': {
    'book': [
      {
        'category': 'reference',
        'author': 'Nigel Rees',
        'title': 'Sayings of the Century',
        'price': 8.95
      },
      {
        'category': 'fiction',
        'author': 'Evelyn Waugh',
        'title': 'Sword of Honour',
        'price': 12.99
      },
      {
        'category': 'fiction',
        'author': 'Herman Melville',
        'title': 'Moby Dick',
        'price': 8.99
      }
    ],
    'bicycle': {
      'color': 'red',
      'price': 19.95
    }
  }
}";

JObject json = JObject.Parse(jsonString);
```

JSONPath Syntax

Basic Selectors

Expression	Description	Example
<code>\$</code>	Root object	<code>\$</code>
<code>.</code>	Current object	<code>\$.store</code>
<code>[]</code>	Array index	<code>\$.store.book[0]</code>
<code>*</code>	Wildcard	<code>\$.store.*</code>
<code>..</code>	Recursive descent	<code>\$..price</code>

Examples with Code

1. Root and Direct Property Access

```
csharp

// Get the entire store object
JToken store = json.SelectToken("$.store");
Console.WriteLine(store);

// Get bicycle price
JToken bicyclePrice = json.SelectToken("$.store.bicycle.price");
Console.WriteLine($"Bicycle price: {bicyclePrice}"); // 19.95
```

2. Array Access

```
csharp

// Get first book
JToken firstBook = json.SelectToken("$.store.book[0]");
Console.WriteLine($"First book: {firstBook}");

// Get last book
JToken lastBook = json.SelectToken("$.store.book[-1]");
Console.WriteLine($"Last book: {lastBook}");

// Get specific book title
JToken title = json.SelectToken("$.store.book[1].title");
Console.WriteLine($"Second book title: {title}"); // Sword of Honour
```

3. Multiple Results with SelectTokens()

```
csharp
```

```
// Get all book titles
IEnumerable<JToken> titles = json.SelectTokens("$.store.book[*].title");
foreach (JToken title in titles)
{
    Console.WriteLine($"Title: {title}");
}

// Get all prices in the store
IEnumerable<JToken> prices = json.SelectTokens("$.price");
foreach (JToken price in prices)
{
    Console.WriteLine($"Price: {price}");
}
```

Advanced JSONPath Features

Array Slicing

```
csharp

// Get first two books
IEnumerable<JToken> firstTwoBooks = json.SelectTokens("$.store.book[0:2]");

// Get books from index 1 to end
IEnumerable<JToken> booksFromOne = json.SelectTokens("$.store.book[1:]");
```

Filtering with Expressions

```
csharp

// Books with price less than $10
IEnumerable<JToken> cheapBooks = json.SelectTokens("$.store.book[?(@.price < 10)]");
foreach (JToken book in cheapBooks)
{
    Console.WriteLine($"Cheap book: {book["title"]}");
}

// Fiction books
IEnumerable<JToken> fictionBooks = json.SelectTokens("$.store.book[?(@.category == 'fiction')]");
foreach (JToken book in fictionBooks)
{
    Console.WriteLine($"Fiction: {book["title"]}");
}
```

Complex Filtering Examples

csharp

```
string complexJson = @"{
  'products': [
    {'name': 'Laptop', 'price': 999, 'inStock': true, 'tags': ['electronics', 'computers']},
    {'name': 'Mouse', 'price': 25, 'inStock': false, 'tags': ['electronics', 'accessories']},
    {'name': 'Book', 'price': 15, 'inStock': true, 'tags': ['education', 'reading']}
  ]
}";

JObject products = JObject.Parse(complexJson);

// Products in stock
var inStockProducts = products.SelectTokens("$.products[?(@.inStock == true)]");

// Products with 'electronics' tag
var electronics = products.SelectTokens("$.products[?(@.tags[*] == 'electronics')]");

// Products with price between 20 and 100
var midRangeProducts = products.SelectTokens("$.products[?(@.price >= 20 && @.price <= 100)]");
```

Working with Results

Converting to Specific Types

csharp

```
// Convert to string
string authorName = json.SelectToken("$.store.book[0].author").ToString();

// Convert to decimal
decimal price = json.SelectToken("$.store.book[0].price").Value<decimal>() ?? 0;

// Convert to custom object
public class Book
{
    public string Category { get; set; }
    public string Author { get; set; }
    public string Title { get; set; }
    public decimal Price { get; set; }
}

JToken bookToken = json.SelectToken("$.store.book[0]");
Book book = bookToken?.ToObject<Book>();
```

Error Handling

csharp

// Safe token selection

```
JToken safeToken = json.SelectToken("$.store.nonexistent?.property");
if (safeToken != null)
{
    Console.WriteLine($"Found: {safeToken}");
}
else
{
    Console.WriteLine("Token not found");
}

// Using null conditional operator
string safeName = json.SelectToken("$.store.book[0].author")?.ToString() ?? "Unknown";
```

Common JSONPath Expressions

csharp

// All authors

```
var authors = json.SelectTokens("$.author");
```

// All items in store (books and bicycle)

```
var allItems = json.SelectTokens("$.store.*");
```

// Books by specific author

```
var melvilleBooks = json.SelectTokens("$.store.book[?(@.author == 'Herman Melville')]");
```

// Expensive items (price > 10)

```
var expensiveItems = json.SelectTokens("$.store.*[?(@.price > 10)]");
```

Performance Tips

1. **Use SelectToken()** for single results instead of SelectTokens().First()
2. **Cache parsed JSON** if querying multiple times
3. **Use specific paths** instead of recursive descent (..) when possible
4. **Validate JSONPath expressions** during development

Complete Example

csharp

```

using System;
using System.Linq;
using Newtonsoft.Json.Linq;

class Program
{
    static void Main()
    {
        string json = @"{
            'users': [
                {'id': 1, 'name': 'John', 'age': 30, 'active': true},
                {'id': 2, 'name': 'Jane', 'age': 25, 'active': false},
                {'id': 3, 'name': 'Bob', 'age': 35, 'active': true}
            ]
        }";

        JObject data = JObject.Parse(json);

        // Get all active users
        var activeUsers = data.SelectTokens("$.users[?(@.active == true)]");
        Console.WriteLine($"Active users: {activeUsers.Count()}");

        // Get user by ID
        var user = data.SelectToken("$.users[?(@.id == 2)]");
        Console.WriteLine($"User 2: {user?["name"]}");

        // Get all ages
        var ages = data.SelectTokens("$.users[*].age");
        var avgAge = ages.Average(age => (int)age);
        Console.WriteLine($"Average age: {avgAge}");
    }
}

```

This tutorial covers the essential JSONPath expressions in Newtonsoft.Json, from basic property access to complex filtering operations. JSONPath provides a powerful way to query and extract specific data from JSON documents efficiently.